AI1804

PROJECT FOR REL301m:

The Chess Strategist

LECTURE: NGUYEN HONG HAI

Team Members

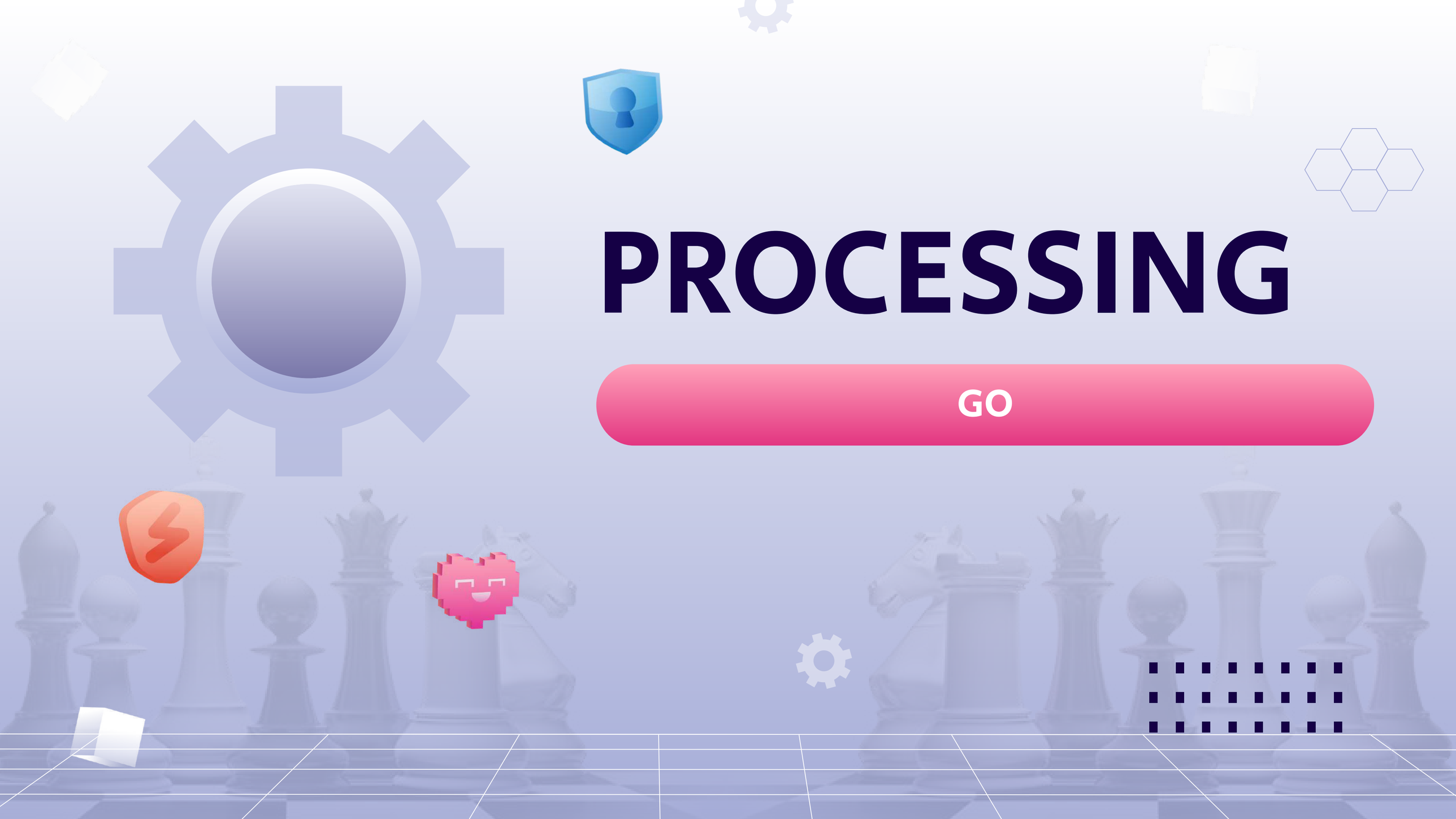
PHAM NGUYEN THANH TAM

LE CAO DANG MINH

HOANG PHUC TRONG

Table of contents

GO


PROCESSING
GO

Building a 9x9 Go AI with 3 Levels

- Project Name: **Go AI (Pygame & Python)**
- Core Technologies: **Pygame, Q-Learning**
- Files for Analysis: **go_game.py, go_engine.py, q_learning.py, reward_logic.py**

System Architecture

- **go_game.py (Core Controller):** Game loop, Graphics (UI/UX), Click handling, Go Logic (capturing, Ko rule).
- **go_engine.py (AI Brain):** Defines the 3 levels: EasyAgent, MediumAgent, HardAgent.
- **q_learning.py (Learning Module):** Provides the generic QLearningAgent Class, manages the Q-Table.
- **reward_logic.py (Reward Logic):** Defines "rewards" for moves (based only on captures).

Initialization & Loading

- User selects [Difficulty (Easy/Medium/Hard)] & [Mode (PVP/AVA)].
- GoGame is initialized.
- _create_agent is called, creating the corresponding Agent.
- If MediumAgent or HardAgent: Load "brain" (Q-table) from a .json file (e.g., data/go_q_table_medium.json).

The Learning Loop (Game Loop)

- The game.run() loop begins.
- When it's the AI's turn (execute_ai_move):
  - Game requests a move from the Agent (agent.choose_move).
  - Agent (Easy, Medium, Hard) returns a move (e.g., (3, 4) or "pass").
  - Game executes the move (place_stone).
  - Game notifies the Agent of the result (agent.learn_from_move).
- End of Game:
  - The Agents (MediumAgent and HardAgent) save their learning progress (agent.save_progress).


METHODS
GO

3 AI Levels (Overview)

- EasyAgent:
  - Method: Random.
  - No learning, no memory, tries not to "pass".
- MediumAgent:
  - Method: Reinforcement Learning (Q-Learning).
  - Self-learning, has memory (Q-Table), 15% exploration (ϵ=0.15$).
- HardAgent:
  - Method: Reinforcement Learning (Q-Learning).
  - Same as Medium, but exploits more, 5% exploration (ϵ=0.05$).

”Easy” Agent

- EasyAgent (Random):
- Priority 1: Get a list of all valid moves (get_valid_moves).
- Priority 2: Filter out the "pass" move (if other moves exist).
- Priority 3: Pick one move randomly from the remaining list.

”Medium” & ”Hard” – Q-Learning

- Technology: Q-Learning (Reinforcement Learning).
- Q-Table: A large dictionary that stores a (Q-Value) for each (State, Action) pair.
- Q(s, a): The "value" of performing action a while in state s.
- State (s): Defined as (Entire board + Current turn).
  - Converted to a string to be used as a "key".
- Action (a): A specific move, represented by a string.
  - Example: "(3, 4)", "pass"

Rewards & Strategy

- Reward (r):
  - Large reward (+10) for each enemy stone captured.
  - Small reward (+0.1) for each valid move (to encourage movement).
- Strategy (ϵ-greedy):
  - Easy: 100% random (no Q-Learning).
  - Medium (ϵ=0.15$): 15% random (Explore), 85% select best move.
  - Hard (ϵ=0.05$): 5% random, 95% select best move (Exploit).

RESULTS

GO

Results & Achievements

- Completed a 9x9 Go game with 3 AI levels.
- EasyAgent: Works as expected (random play), serves as a baseline for comparison.
- Medium/Hard Agent:
  - Can self-learn after each move (learn_from_move).
  - Has a self-training mode (AVA - AI vs. AI) for data generation.
  - Learning progress is saved (save_progress) and reused (load_q_table).
- Clear UI, with last-move highlighting and info display (last_move).

Limitations & Future Development

Limitations:

- State Explosion:
  - The Q-Table becomes too large, learns very slowly, and uses too much memory.
  - The AI cannot "generalize" (fails to recognize similar positions).
- Simple Rewards:
  - The AI is only programmed to capture stones.
  - It doesn't understand core strategic concepts: Territory, Life/Death Influence.

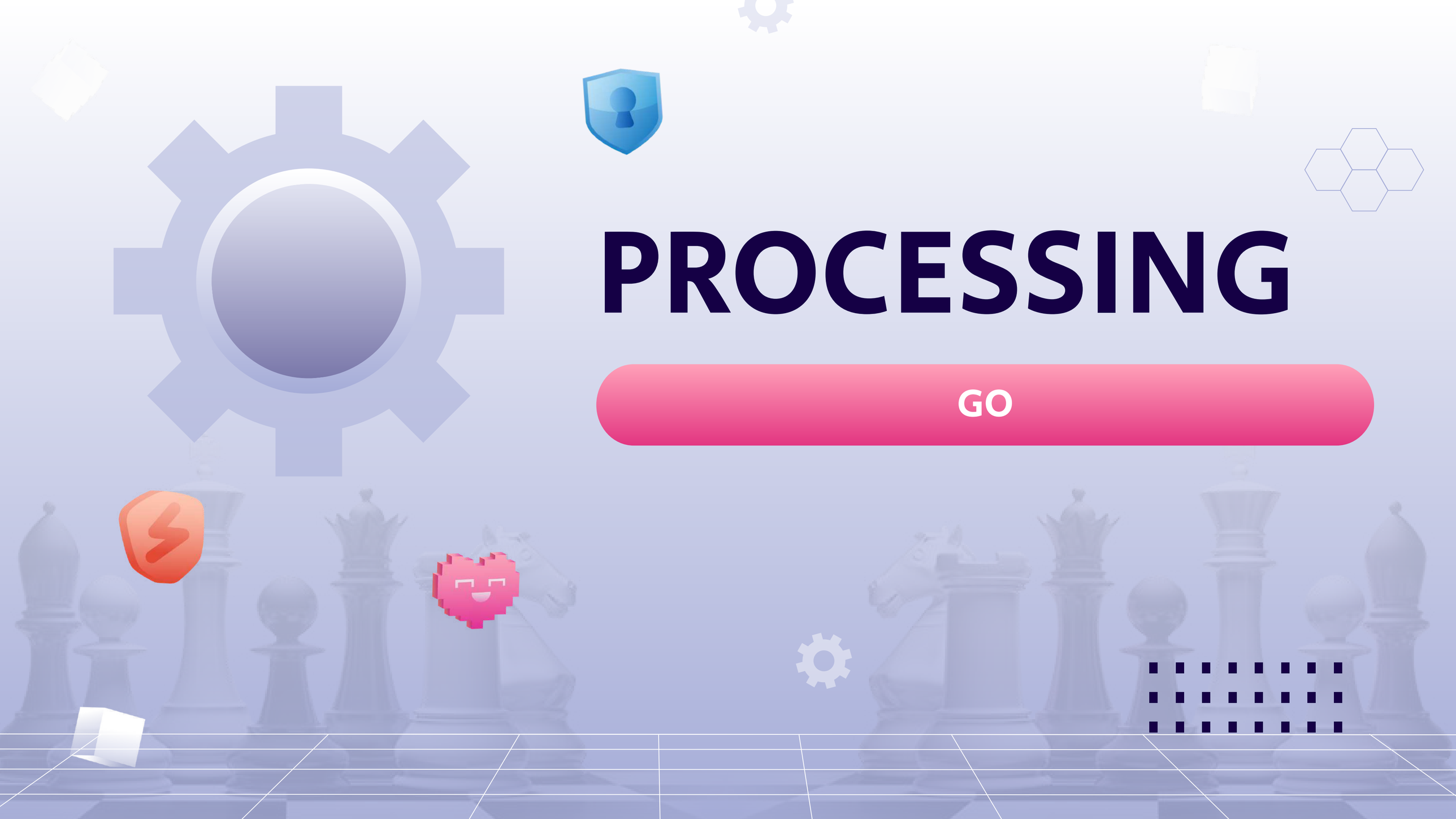
Limitations & Future Development

Future Development:

- Use a Deep Q-Network (DQN)(Neural Network) instead of a Q-Table to allow "generalization".
- Improve the Reward Logic to include points for securing territory.

07

CHESS


PROCESSING
GO

Building an Intelligent Chess AI with 3 Levels

- Project Name: **Chess AI (Pygame & Python)**
- Core Technologies: **Pygame, python-chess, Q-Learning, Stockfish Engine**
- Files for Analysis: **chess_game.py, chess_engine.py, q_learning.py, reward_logic.py**

System Architecture

- **chess_game.py (Core Controller):** Game loop, Graphics (UI/UX), Time management, Click handling.
- **chess_engine.py (AI Brain):** Defines the 3 levels: EasyAgent, MediumAgent, HardAgent.
- **q_learning.py (Learning Module):** Provides the generic QLearningAgent Class, manages the Q-Table.
- **reward_logic.py (Reward Logic):** Defines "rewards" and "penalties" for moves.

Initialization & Loading

- User selects [Difficulty (Easy/Medium/Hard)] & [Mode (PVP/AVA)].
- ChessGame is initialized.
- _create_agent is called, creating the corresponding Agent.
- If MediumAgent, load "brain" (Q-table) from data/chess_q_table.json.
- If HardAgent, initialize and connect to the Stockfish Engine.

The Learning Loop

- The game.run() loop begins.
- When it's the AI's turn:
  - Game requests a move from the Agent (agent.choose_move).
  - Agent (Easy, Medium, Hard) returns a move.
  - Game executes the move (board.push(move)).
  - Game notifies the Agent of the result (agent.learn_from_move).
- End of Game:
  - The Agent (MediumAgent only) saves its learning progress (agent.save_progress).


METHODS
GO

3 AI Levels (Overview)

- EasyAgent:
  - Method: Simple Heuristics (Rules).
  - No learning, no memory.
- HardAgent:
  - Method: External Engine Integration.
  - Uses Stockfish Engine (one of the strongest AIs).
  - Can limit ELO (strength).
- MediumAgent:
  - Method: Reinforcement Learning (Q-Learning).
  - Self-learning, has memory (Q-Table), and combines Heuristics.

Methods: ”Easy” & ”Hard” Agents

- EasyAgent (Heuristics):
  - Priority 1: Find all Captures. Pick one randomly.
  - Priority 2: If no captures, find all Checks. Pick one randomly.
  - Priority 3: If neither, pick a random legal move.
- HardAgent (Stockfish):
  - Initialize chess.engine.SimpleEngine.popen_uci(STOCKFISH_PATH).
  - Configure strength (Limit ELO or Full-power).
  - Simply call self.engine.play(board, ...) to get the best move.

”Medium” – Q-Learning

- Technology: Q-Learning (Reinforcement Learning).
- Q-Table: A large dictionary that stores a (Q-Value) for each (State, Action) pair.
- Q(s, a): The "value" of performing action a while in state s.
- State (s): Defined as the first 3 parts of the FEN string (Piece positions + Turn + Castling rights).
  - **Example: "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq"**
- Action (a): A specific move, represented by a UCI string.
  - **Example: "e2e4", "g1f3"**

”Medium” – Rewards

- The Agent learns via "rewards" (reward_logic.py).
- VERY LARGE Reward:
  - +1000 points for Checkmate.
- LARGE Rewards (Captures):
  - +90 (capture Queen), +50 (capture Rook), +30 (capture Knight/Bishop), +10 (capture Pawn).
- SMALL Reward:
  - +5 points for a Check.
- Penalty:
  - -0.1 points for every normal move (to avoid wasteful moves).
- Draw: 0 points.

”Medium” – Hybrid Strategy

- MediumAgent doesn't just use Q-Learning.
- This is a "Hybrid" strategy.
- When choosing a move:
  - It gets the best move from the Q-Table (Exploit) AND a random move (Explore - Epsilon).
  - BUT: It also finds all Captures and Checks.
  - There is a 40% chance to "override" Q-Learning and pick a random Capture.
  - There is a 30% chance to "override" and pick a random Check.
  - Otherwise, it uses the move from Q-Learning.

RESULTS

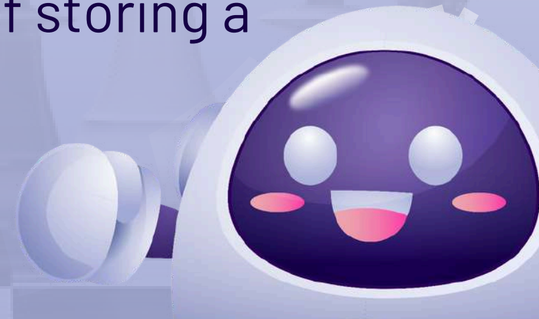
GO

Results & Achievements

- Completed a Chess game with 3 distinct AI levels.
- EasyAgent: Works as expected, plays naively, easy to defeat.
- HardAgent: Works powerfully, a nearly unbeatable opponent (at Full-power).
- MediumAgent:
  - Can self-learn after each move (learn_from_move).
  - Has a self-training mode (AVA - AI vs. AI) for data generation.
  - Learning progress is saved and reused (AI gets smarter after each session).
- Smooth UI, with last-move highlighting, easy to follow.

Limitations & Future Development

- State Explosion:
  - Using the first 3 FEN parts as a 'key' still creates billions of states.
  - The Q-Table becomes too large, uses too much memory, and learns very slowly.
  - The AI cannot 'generalize' (e.g., it doesn't recognize 2 similar positions).
- Simple Rewards:
  - The AI is only programmed to capture pieces and check.
  - It doesn't understand strategic concepts like: Center Control, Pawn Structure, or King Safety.
- Future Development:
  - Use a Neural Network (Deep Learning) to estimate Q-Values instead of storing a Q-Table (called a Deep Q-Network - DQN).
  - Improve the reward logic, add points for strategic positions.



08

XIANGQI

PROCESSING

System Architecture

- xiangqi_game.py handles the game loop, the UI/UX, and event handling.
- xiangqi_engine.py is the brain, containing the 3 AI levels.
- reward_logic.py is used to define the rewards for the AI.
- And xiangqi.py is the core logic file, handling all the rules of movement, checkmate, and win/loss conditions in Xiangqi.

Initialization & Loading

- The game process begins when the user selects the [Difficulty] and [Game Mode].
  - The system will initialize the game (XiangqiGame).
  - A corresponding AI Agent (Easy, Medium, or Hard) is created.
  - EasyAgent will play using simple heuristic rules.
  - MediumAgent and HardAgent will use Q-Learning.
  - The Pygame interface is also drawn, displaying the board, a 5-minute countdown clock for each side, and highlighting the last move.

The Game Loop

- In the game loop, when it's the AI's turn:
  - It will select a valid move.
  - The game executes that move, updates the clock, highlights the move, and switches turns.
  - When time runs out or there's a checkmate, the game ends and displays the winner.
  - Notably, if it's a MediumAgent, it will execute learn_from_move (learn fro move) and save_progress (save its learning) at the end.

METHODS

3 AI Levels

- The project features 3 AI levels with very different methods:
  - EasyAgent: Uses simple heuristics. It's easy to beat and does not learn.
  - MediumAgent: Uses Q-Learning. It can self-learn from rewards, knows to prioritize captures and checks.
  - HardAgent: (By design) will use Deep Reinforcement Learning – combining Neural Networks and Monte Carlo Tree Search (MCTS) – for deep strategic thinking.

EasyAgent – Heuristic Rules

- The logic for EasyAgent is very simple:
  - Priority 1: Find moves that capture a piece (Rook, Horse, Cannon...).
  - Priority 2: If no capture, find a move that Checks the General.
  - Priority 3: If still nothing, it picks a random valid move.
  - It accurately simulates a novice player and does not get smarter over time.

MediumAgent — Q-Learning

- MediumAgent is the focus of the machine learning aspect.
  - It uses Q-Learning.
  - The State is encoded from the board position (usually a FEN string or a 10x9 matrix).
  - The Action is a single move.
  - It learns by updating a Q-Table (a value lookup table) based on the Bellman equation, to optimize future rewards.

HardAgent – Deep Reinforcement Learning

- HardAgent is the highest tier. It combines a Deep Neural Network with Monte Carlo Tree Search (MCTS).
- A Policy Network predicts which moves are good.
- A Value Network evaluates the current board position.
- MCTS simulates thousands of moves to select the one with the highest value. It can self-train via self-play and has strategic thinking, looking 5-8 moves ahead.

Reward

- For the MediumAgent to learn, we must define Rewards:
  - Capturing a strong piece (Rook) gets the highest reward: +70.
  - Capturing a Cannon or Horse: +50.
  - Capturing a Pawn: +10.
  - Checking the General gives +30, and Checkmate is the final goal, rewarding +1000.
  - Conversely, being checked or losing the General is a heavy penalty of -1000.
  - And to avoid wasteful moves, every normal move gets a slight penalty of

RESULTS

Results & Achievements

- Completed a Pygame interface for Xiangqi with a 5-minute clock.
- Three functional AI levels, where MediumAgent can self-learn from rewards.
- Support for both Player vs. AI (PvP) and AI vs. AI (AvA) (which is used for self-training).
- In testing (for example), the MediumAgent wins against the EasyAgent about 78% of the time after training.
- Most importantly, the code architecture is easily expandable to integrate the HardAgent (Deep RL) in the future.

Limitations & Future Development

Limitations:

- Q-Table Explosion: The MediumAgent's Q-Table becomes massive, consumes memory, and cannot "generalize" (it doesn't recognize similar positions).
- Simple Rewards: The current rewards only focus on "capturing pieces," lacking strategic elements like "position" or "defense."
- Missing Data: There is no opening book or endgame tablebase.

Limitations & Future Development

Future Development:

- Short-term: Improve the reward logic (add positional points) and add a simple opening book.
- Long-term: Completely replace the Q-Table with a Neural Network (DQN) or a full AlphaZero pipeline (DNN + MCTS) for the HardAgent, and build a benchmarking system to measure performance accurately

Thanks!